# CTI Entity Attribution with Small Open-Source LLMs

**System Architecture & Design Document — CS 6338 Knowledge Engineering, Milestone 2**

Shinhyuk Park · October 17, 2026

## 1. Executive Summary & System Requirements

This project builds a knowledge-graph-guided attribution engine that helps a small open-source LLM (Llama-3-8B-Instruct) map threat-report passages to MITRE ATT&CK techniques (CTIConnect task ATA) and CWE weaknesses (task VCA). On CTIConnect, LLaMA-3-8B reaches only 0.28 F1 on ATA and 0.40 on VCA, against 0.90 and 0.76 for GPT-5; the benchmark authors identify attribution as the one task family bottlenecked by model capability rather than retrieval.

**Scope change (approved by Dr. Dogdu, Oct 16, 2026).** The proposal's four-task, multi-hop GraphRAG evaluation is narrowed to entity attribution only. Entity-linking tasks (RCM, ESD, ATD) already reach about 0.97 F1 with domain-specific retrieval and are single-hop, so they cannot show what a graph adds. Multi-hop questions are no longer required.

**Research question.** Can a cybersecurity knowledge graph, used as a reasoning scaffold, close the gap between small open-source models and frontier models on CTI entity attribution?

**Domain and persona.** A SOC or threat-intelligence analyst reads a vendor report and needs the ATT&CK technique or CWE weakness a passage describes, with the reasoning and the KB entry that support it. The system runs its answering model locally (Ollama on a MacBook), so reports are never sent to a commercial LLM.

### Functional requirements

| ID | Requirement |
|---|---|
| FR1 | For a report passage, return exactly one ATT&CK technique ID (ATA) or CWE ID (VCA), the benchmark's single-gold format, and only an ID that exists in the graph |
| FR2 | Cite the evidence: the candidate set, the chosen entry's KB text and the graph path behind it (passage → CAPEC/CVE → CWE, or report example → technique) |
| FR3 | Use only CTIConnect's `corpus_kb` snapshot and report summaries; never filter the answer space by domain or retired status |
| FR4 | Log every LLM call (prompt, output, tokens, latency) and every retrieved candidate |

### Non-functional requirements

| ID | Requirement | Target |
|---|---|---|
| NFR1 | Local inference | Llama-3-8B-Instruct fp16 in Ollama on a MacBook; no report text sent to a hosted LLM |
| NFR2 | Latency per question | under 10 s for one-call methods; under 30 s with VCA's five rotation calls |
| NFR3 | Reproducibility | temperature 0, pinned dataset commit, model digest recorded per run |
| NFR4 | Cost and integrity | embeddings cached on disk (one-time KB embedding about $0.09); test split never used during development |

## 2. End-to-End System Architecture

The system has an offline build stage that turns official records into a validated Neo4j graph, and a per-question stage in which the graph produces candidates and the small LLM only chooses among them.

### Data flow

| From → To | Input → output | Protocol |
|---|---|---|
| Snapshot → loader, extractor | `corpus_kb` JSONL and report summaries → validated nodes, edges and report layer | file read with sha256 check; OpenAI chat API in JSON mode for extraction |
| Loader, embedders → Neo4j | records, rule results, text and graph vectors → graph with constraints and vector indexes | Bolt (neo4j driver), batched `UNWIND` writes |
| Chatbot → FastAPI | passage, taxonomy, optional question → `AttributionRequest` | HTTP POST `/attribute` |
| Retrieve, expand ↔ Neo4j | question and passage vectors → top-5 candidates with evidence cards (R4–R6) | Cypher: vector similarity, one-hop aggregation |

| From → To | Input → output | Protocol |
|---|---|---|
| Select → Ollama | 5 cards → one ID (VCA: majority over 5 rotations) | OpenAI-compatible chat API, JSON-schema enum, temperature 0 |
| Answer → chatbot, runner → scorer | chosen ID, path, calls → `AttributionResponse`; P / R / F1 | HTTP response; `predictions.jsonl`; CTIConnect `metrics.py` |

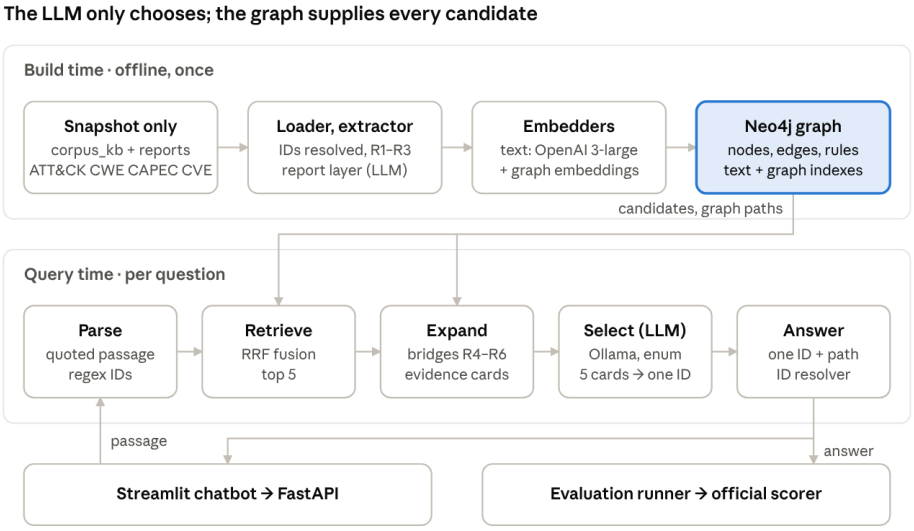



Figure 1. System architecture: build time and query time.

Every ID the system can answer was written by the loader from an official record; the LLM's output is restricted to IDs the graph handed it.

## 3. Knowledge-Graph Schema & Construction

The graph is built only from CTIConnect's `corpus_kb` snapshot (commit 554797d) and its 321 report summaries. Structure the snapshot already records (ATT&CK hierarchy and tactics, CAPEC links to CWE and ATT&CK, CVE weakness labels) is parsed deterministically with no LLM, and its edge counts match CTIFoundry's for the same snapshot. Outside data (ATT&CK STIX, CWE XML, NVD) and the benchmark's `construction/` files, which generated the questions and their answers, are excluded so results stay comparable with the paper; as a consequence the graph has no CWE parent–child hierarchy.

### Node types

| Label | Key | Attributes | Nodes |
|---|---|---|---|
| `AttackTechnique` | `id` (T1059.001) | title, contents, tactics, platforms, domain, replaced_by | 1,076 |
| `Tactic` | `name` (defense-evasion) | — | 19 |
| `CWE` | `id` (CWE-79) | title, contents (empty for 412) | 1,342 |
| `CAPEC` | `id` (CAPEC-66) | name, description | 615 |
| `CVE` | `id` (CVE-2012-4687) | description | 2,751 with a description and a snapshot CWE |
| `ThreatReport` | `id` (BLOG-10) | title, url, published, vendor | 321 |
| `ReportChunk` | `id` (BLOG-10#2) | text, start, end | 650 |
| `ReportBehavior` | `id` | verbatim behavior phrase | 3,512 |
| `Passage` | question id | text, report_id (evaluation only) | 379 |

### Relationship predicates

| Predicate | From → To | Derived from | Edges |
|---|---|---|---|
| `SUBTECHNIQUE_OF` | AttackTechnique → AttackTechnique | ID pattern (T1059.001 → T1059) | 518 |
| `IN_TACTIC` | AttackTechnique → Tactic | `kill_chain_phases` | 1,374 |
| `REPLACED_BY` | AttackTechnique → AttackTechnique | retired-ID rule R2 (Section 4) | 103 |
| `EXPLOITS_WEAKNESS` | CAPEC → CWE | CAPEC `Related_Weaknesses` | 1,212 |
| `MAPS_TO_TECHNIQUE` | CAPEC → AttackTechnique | CAPEC `Taxonomy_Mappings` | 271 |
| `HAS_WEAKNESS` | CVE → CWE | CVE `weaknesses` | 3,286 |
| `HAS_CHUNK`, `HAS_BEHAVIOR` | ThreatReport → ReportChunk → ReportBehavior | report-layer chunker and extractor | 650, 3,512 |
| `GROUNDS` | ReportBehavior → AttackTechnique | report-layer extractor, validated | 3,512 |
| `DESCRIBES_TECHNIQUE`, `DESCRIBES_WEAKNESS` | Passage → AttackTechnique / CWE | CTIConnect gold, **evaluation only** | 379 |

The gold `DESCRIBES_*` edges live in a separate evaluation database and are never loaded into the graph the engine queries.

### LLM extraction: the report layer

Prose is the only input that needs an LLM. The report summaries become labelled examples of report wording for each technique, following CTIFoundry's build phase (Steps 0 and 2; its entity-mention and resolution steps serve multi-document synthesis, which is out of scope).

- **Chunking**: clause-bounded chunks of 200–900 characters, split after `.`, `!`, `?` or `;`, with no overlap. Each chunk keeps exact character offsets into its report, re-verified on load (650 chunks).
- **Prompt**: CTIFoundry's TTP-grounding prompt (App. G.3) verbatim, with the tactic list filled from the snapshot. It carries seven few-shot mappings from outcome phrases to techniques ("encrypts files" → T1486) and rules that exclude defenses, generic outcomes and bare tool names. Model gpt-5.4-mini-2026-03-17 in JSON mode; output `{"ttps": [{"behavior", "technique_id", "technique_name"}]}`.
- **Error handling**: an unparseable response yields no records for its chunk. A record is kept only if its ID exists in the snapshot and its behavior phrase occurs verbatim in its chunk; 3,512 of 3,741 extracted records pass.
- **Confidence**: the prompt tells the model to omit behaviors it is unsure of, and both checks are hard gates. A graded score, the cosine between a behavior and its technique's text with a threshold set on the 69 gold triples, is the planned extension.
- **Question side**: the quoted passage and any CVE, CWE or ATT&CK IDs are taken from each question by regex, never by the LLM.

### Entity resolution: names to IDs

Small models recall titles more reliably than numbers, so a generated ID is never trusted on its own:

1. **Exact match**: the ID exists in the graph and its title matches the generated name after normalization.
2. **Title lookup**: normalized title equality within the target taxonomy.
3. **Retired IDs**: follow `REPLACED_BY` to the current sub-technique.
4. **Embedding fallback**: nearest title by cosine similarity, accepted only above a threshold $\tau$ tuned on the dev split (initial $\tau = 0.80$); below $\tau$ the answer is marked unresolved.

## 4. Graph Storage, Enhancement & Reasoning

The graph lives in Neo4j 5, which holds the typed nodes and edges and vector indexes over every text the engine searches. The reasoning layer adds three bridges from report wording to taxonomy entries (CAPEC patterns, CVE examples and report-layer behaviors) and derives domain, retired status and tactics as attributes. It never filters candidates by those attributes, because a filter would assume what the answers look like.

### Database initialization (`schema.cypher`)

```
// schema.cypher (excerpt): 8 uniqueness constraints, 4 lookup indexes, 5 vector indexes
CREATE CONSTRAINT technique_id IF NOT EXISTS FOR (t:AttackTechnique) REQUIRE t.id IS UNIQUE;
// ... the same for Tactic.name and the CWE, CAPEC, CVE, ThreatReport, ReportChunk, ReportBehavior ids
CREATE INDEX technique_title IF NOT EXISTS FOR (t:AttackTechnique) ON (t.title_norm);
// ... cwe_title, technique_domain, chunk_report
CREATE VECTOR INDEX technique_emb IF NOT EXISTS FOR (t:AttackTechnique) ON (t.embedding)
OPTIONS {indexConfig: {`vector.dimensions`: 3072, `vector.similarity_function`: 'cosine'}};
// ... cwe_emb, capec_emb, cve_emb, behavior_emb with the same settings

// the CAPEC bridge (rule R4) for one passage vector $v, exact similarity
MATCH (c:CAPEC) WITH c, vector.similarity.cosine(c.embedding, $v) AS score
ORDER BY score DESC LIMIT 10
MATCH (c)-[:EXPLOITS_WEAKNESS]->(w:CWE)
RETURN w.id AS cwe, sum(2 * score - 1) AS bridge_score ORDER BY bridge_score DESC;
```

The report layer is stored as `(ThreatReport)-[:HAS_CHUNK]->(ReportChunk)-[:HAS_BEHAVIOR]->(ReportBehavior)-[:GROUNDS]->(AttackTechnique)`. `scripts/load_graph.py` applies `schema.cypher` (17 statements) and loads 10,286 nodes, 9,296 of them with embeddings, from the on-disk embedding cache in about 30 seconds, checking every edge count against the Python derivation. Neo4j 5.26 runs locally in Docker.

SGS can take its candidates from Neo4j (`--backend neo4j`): each bridge is one Cypher query. Evaluation uses exact `vector.similarity.cosine`, so the Cypher path returns the same rankings as the in-memory reference up to floating-point near-ties; the HNSW vector indexes serve the chatbot, where approximate search is acceptable.

### Reasoning layer: snapshot-derived rules

| Rule | Inference | How SGS uses it | Design rationale |
|---|---|---|---|
| R1 Domain | Mobile if platforms include Android or iOS, ICS if the ID starts with T0, else Enterprise | platforms shown on ATA cards; never a filter | a domain filter would assume the answer space |
| R2 Retired ID | an Enterprise parent-level ID with the same title as an Enterprise sub-technique → `REPLACED_BY` (103 IDs) | stored; not used to filter | resolves old IDs in generated text; filtering them is an ablation only |
| R3 Hierarchy and tactics | `SUBTECHNIQUE_OF` from the ID pattern, `IN_TACTIC` from `kill_chain_phases` | tactics shown on ATA cards | tactic-first narrowing rejected: in pilot runs the 8B model named the tactic too unreliably to prune on it |
| R4 CAPEC bridge | passage ≈ CAPEC description → `EXPLOITS_WEAKNESS` → CWE | VCA candidates; CAPEC name on VCA cards | CAPEC descriptions use attack language close to report wording |
| R5 CVE examples | passage ≈ CVE description → `HAS_WEAKNESS` → CWE vote | VCA candidates; closest CVE on VCA cards | 2,751 CVE descriptions are labelled examples of 260 CWEs |
| R6 Report examples | passage ≈ report-layer behavior from another report → technique vote | ATA candidates; 2 examples on ATA cards | other reports' wording is closer to a passage than ATT&CK's own definitions |

R1–R3 are materialized as properties and edges at load time; R4–R6 run per question as a vector search followed by one-hop aggregation.

### Gold-triple verification (`scripts/verify_triples.py`)

The engine's answers are scored as triples against `data/gold_triples.json`: 69 triples (28 `DESCRIBES_TECHNIQUE`, 41 `DESCRIBES_WEAKNESS`), one per dev-split question, from CTIConnect's expert-verified answers.

The script reads a run's `predictions.jsonl` and extracts IDs with CTIConnect's official `extract_ids` / `normalize_id`, so triple scores agree with benchmark scores. A predicted triple counts when (report, predicate, ID) equals a gold triple exactly; precision, recall and F1 are reported overall and per predicate. Near misses (the gold technique's parent or child), retired IDs and IDs absent from the snapshot are counted separately, never as hits.

## 5. Featurization, Embedding & Hybrid Indexing

Candidate generation is the first bottleneck: the small model cannot choose an answer it never sees. This section specifies the text features, the snapshot bridges, two graph embeddings and the indexes, and how their rankings are fused. Every setting is chosen on the dev split by candidate recall@5, which needs no LLM calls; the test split is never used for tuning.

### Node featurization

Each searchable node gets one text view from its own snapshot fields, embedded with text-embedding-3-large (E, 3,072 dimensions) and L2-normalized: title + `contents` for techniques and CWEs, name + `Description` for CAPEC patterns, the description for CVEs and the phrase itself for report behaviors.

### Scoring functions

With e(x) = E(x) / ‖E(x)‖, a dense ranking orders nodes v by s(q, v) = e(q) · e(v). The three bridges score a target t by the similarity of the records that point to it:

$$S_{\mathrm{B}}(t) = \sum_{x \,\in\, \mathrm{top}_{n_B}(\mathrm{B},\, q),\; x \to t} s(q, x), \qquad B \in \{\mathrm{CAPEC}\ (n=10),\ \mathrm{CVE}\ (n=20),\ \mathrm{Report}\ (n=30)\}$$

Report behaviors from the question's own source report are excluded. Rankings are fused by reciprocal-rank fusion:

$$\mathrm{RRF}(v) = \sum_{r \in R} \frac{1}{60 + \mathrm{rank}_r(v)}$$

### Queries and chosen configurations

Two queries are built per question: the full question q_Q and its quoted passage q_P. The passage alone retrieves worse than the full question, so the full question stays in every configuration.

- **ATA**: RRF of [RRF(dense(q_Q), dense(q_P)) and S_Report(q_P)] over all 1,076 techniques.
- **VCA**: RRF of dense(q_Q) over CWE records, S_CAPEC(q_P) and S_CVE(q_P).

**Considered and dropped.** BM25 keyword search and, for ATA, the CAPEC bridge (only 177 CAPECs map to ATT&CK) lowered fused recall in pilot measurements. Domain and retired-ID filters are left out because they encode an assumption about the answers. Re-ranking VCA candidates by their single closest CVE example lost to the CVE vote.

### Graph representation learning

Two graph embeddings are specified over the snapshot graph G = (V, E) of Section 3: 9,315 nodes (techniques, tactics, CWEs, CAPECs, CVEs, report behaviors) and 10,276 undirected edges. The edge set is a design choice: taxonomy (`SUBTECHNIQUE_OF`, `REPLACED_BY`, `IN_TACTIC`), links (`EXPLOITS_WEAKNESS`, `MAPS_TO_TECHNIQUE`, `HAS_WEAKNESS`) and report (`GROUNDS`).

**A. Propagated text embeddings** (training-free GraphSAGE-mean, APPNP form). Each node starts from its text embedding x_v = e(text(v)) and mixes in its neighbors' vectors:

$$H^{(0)} = X, \qquad H^{(k+1)} = (1-\alpha)\, X + \alpha\, D^{-1} A\, H^{(k)} \quad (k = 0, \ldots, K-1), \qquad h_v = \frac{H_v^{(K)}}{\| H_v^{(K)} \|}$$

A is the adjacency over the chosen edge set and D its degree matrix; isolated nodes keep x_v. The vectors stay in the text space, so a question is scored directly: s_A(q, v) = e(q) · h_v. One hop folds a CWE's CVE examples and CAPEC patterns into it, and a technique's parent, sub-techniques, tactics and report examples.

**B. Text + node2vec.** A structural embedding g_v of 128 dimensions is learned by node2vec: second-order random walks with return parameter p and in-out parameter q, then skip-gram with negative sampling over each node's walk context N_W(v):

$$\max_{g} \sum_{v \in V} \sum_{u \in N_W(v)} \Big[ \log \sigma(g_u^\top g_v) + \sum_{j=1}^{5} \mathbb{E}_{w \sim P_n} \log \sigma(-g_w^\top g_v) \Big]$$

A question is not a node, so it is anchored through its 10 nearest KB nodes by text, and the two similarities are added:

$$g_q = \text{normalize}\Big( \sum_{i \in \text{NN}_{10}(q)} \max(s(q,i),0)\, g_i \Big), \qquad s_B(q,v) = s(q,v) + \lambda\, g_q^\top g_v$$

**Leakage guard.** Report-behavior edges from the question's own report are removed before propagation; during evaluation the vectors are recomputed per held-out report, while the deployed chatbot uses the full graph. node2vec is trained on the `corpus_kb` edges only, without report behaviors.

**Use in the methods.** A graph-embedding ranking scores targets for the question and for the passage, fused by RRF like the other rankings. It enters two evaluation rows (Section 7): GE-RAG, vanilla RAG's prompt over the GE top 5, and SGS + GE, the GE ranking added to SGS's fusion.

| Parameter | Values searched | Selection |
|---|---|---|
| A: edge set | taxonomy, links, kb (taxonomy + links), report, kb + report | dev recall@5, one setting shared by ATA and VCA |
| A: mixing α, hops K | α ∈ {0.2, 0.4, 0.6}, K ∈ {1, 2} | dev recall@5 |
| B: walks | 10 walks per node, length 40, p = 1, q ∈ {0.5, 1, 2} | dev recall@5 |
| B: skip-gram | 128 dimensions, window 5, 5 negative samples, 1 epoch (gensim 4.4) | fixed |
| B: weight λ | {0.1, 0.25, 0.5, 1.0} | dev recall@5 |

**Indexing.** Propagated vectors (3,072-d) for techniques and CWEs and node2vec vectors (128-d) are stored as node properties with their own Neo4j vector indexes (`technique_prop_emb`, `cwe_prop_emb`, `node2vec_emb`), planned additions to `schema.cypher` beside its five text indexes. Report text is indexed at the behavior level: each ReportBehavior is a verbatim span of its ReportChunk, and the behavior index serves as the chunk index because one 900-character chunk mixes several behaviors. Code: `src/ctiattr/graph_embed.py`; search grid: `scripts/graph_embed_recall.py`.

### Hyperparameters

Fixed: text-embedding-3-large (3,072 dimensions, L2-normalized, as in CTIConnect) and the standard RRF constant 60. Set on the dev split: 10 / 20 / 30 neighbours for the CAPEC, CVE and report bridges, and 5 candidates shown to the model, since 10 lowered selection accuracy in pilot runs even when the gold was listed.

## 6. Attribution QA Engine & Prompt Templates

The method, Snapshot-Guided Selection (SGS), gathers 5 candidates and their evidence from the snapshot graph and lets the small model make one constrained choice among them. Its configuration was frozen on the dev split and is implemented in `src/ctiattr/sgs.py`; the baselines reuse CTIConnect's prompts verbatim, so the method is the only difference.

### Baselines

| Method | Steps | Source |
|---|---|---|
| CB: closed-book | question → answer | CTIConnect `closed_book`, prompts verbatim |
| VR: vanilla RAG | embed full question → top-5 raw KB records (600 characters each) → answer | CTIConnect `vanilla_rag`, prompts verbatim |
| DtR (reference row) | decompose → retrieve per behavior → aggregate | CTIConnect released code; not reproducible as described in the paper (Section 7) |

The VR answer prompt is CTIConnect's, unchanged: the question, the retrieved records, then "Based on the candidates above, answer the question. Cite the single best-matching identifier explicitly."

### SGS procedure (frozen)

1. **Parse.** Extract the quoted passage and any CVE, CWE or ATT&CK IDs by regex.
2. **Candidates.** Run the chosen configuration of Section 5 and keep the top 5.
3. **Evidence cards.** One line per candidate, built only from the snapshot:

- ATA: `ID | title | tactics | platforms | detection: <first sentence> | report examples: "<snippet>"; "<snippet>"`, the two report-layer sentences closest to the passage, never from its own report.
- VCA: `ID | title | consequences | related attack pattern: <CAPEC name> | example vulnerability: <closest CVE description>`.

4. **Choice.** The model returns one ID; JSON-schema structured output restricts it to an enum of the 5 candidate IDs, so an invented ID is impossible.
   - ATA: one call, candidates in retrieval order.
   - VCA: five calls over the cyclic rotations of the 5 candidates (each appears once in every position), majority vote, ties to the better retrieval rank. This cancels the position bias seen in pilot runs, where the 8B model favored early list positions.
5. **Answer.** Exactly one ID plus its title. A final check confirms the ID is one of the listed snapshot IDs and the title contains no other identifier, since the official scorer counts every ID in the text.

### Prompt (exact)

System: `You are a cyber threat intelligence analyst.` Output schema: `{"id": enum of the candidate IDs}`.

```
You are mapping a threat-report passage to {MITRE ATT&CK techniques | CWE weaknesses}.

Passage:
"{passage}"

Candidates:
{cards}

Choose the ONE candidate that best describes the behavior in the passage. Judge by
what each candidate means, not by shared words. Return JSON with its id.
```

**Answer generator prompt (exact).** After the choice, a second call writes the answer the chatbot shows and RAGAS scores. Any other ATT&CK, CWE or CAPEC identifier in its output is replaced by that entry's title, so official F1 is unchanged.

```
The passage below was attributed to {id} — {title}.
Evidence: {card}
Passage: "{passage}"
Answer the user's question in 3–5 sentences: {question}
Mention no other ATT&CK, CWE or CAPEC identifier.
```

**Alternatives considered.** Ten candidates instead of five lowered selection accuracy even when the gold was listed. A pairwise tournament needed up to 8 calls with no gain over one listwise call, and a rotation vote for ATA gained nothing once report examples were on the cards. Tactic-first narrowing is covered under R3, and an output fact-checker is unnecessary because the enum restriction and the final check already block invented IDs.

## 7. Evaluation Pipeline & Benchmark Protocol

All methods are scored with CTIConnect's official identifier F1 on its 379 attribution questions (ATA 160, VCA 219), with the same model, embedder and data, so differences come from the method alone. Configurations are chosen on the 73-question dev split; the 306-question test split stays untouched until the final evaluation, where each method runs on it once.

### Data and splits

- **Questions**: CTIConnect `data/entity_attribution`, pinned to commit 554797d. Every question has exactly one gold ID.
- **Split by report**: questions are split by source report (seed 13), so no report's passages appear on both sides: dev 73 questions from 29 reports (30 ATA, 43 VCA), test 306.

- **Quick-look set** (`eval_queries.json`): 20 dev questions (10 ATA, 10 VCA), for fast checks and the human evaluation. They are single-hop (passage → one ID) by design, under the approved scope change (Section 1).
- **Known ceiling**: two pairs of ATA questions are word-for-word identical but have different gold IDs (e.g. ata-106 T1105, ata-107 T1059), so maximum ATA F1 is 0.988.

### Automated metrics

| Metric | What it measures | Implementation |
|---|---|---|
| Identifier P / R / F1, exact match, IDs per answer | the benchmark score and over-prediction | CTIConnect `evaluation/metrics.py`, vendored unmodified; its 21 tests pass |
| Candidate recall@k (k = 1, 5, 10, 20, 50) | whether the gold ID reaches the model | `scripts/retrieval_recall.py` |
| Triple P / R / F1 | `DESCRIBES_*` triples vs 69 gold triples | `scripts/verify_triples.py` (Section 4) |
| RAGAS faithfulness, answer relevancy, context recall | whether the answer is supported by the KB entries shown | RAGAS 0.4 on the 20 eval queries (configuration below) |

Scoring rule: a failed call is scored as an empty answer (F1 = 0), never dropped. CTIConnect's own aggregator drops missing predictions, which would inflate scores.

### Models and comparison

- **Answering model**: Llama-3-8B-Instruct only, the smallest model in CTIConnect's Table III, served by Ollama at full precision (fp16), temperature 0. Restricting the study to one model keeps every comparison on the same footing within the project's time.
- **Compared methods**: CB and VR (CTIConnect's baselines), SGS (Section 6), and the two graph-embedding rows of Section 5: GE-RAG and SGS + GE. Each pair differs in one component, so the comparison isolates it.
- **DtR as a reference row only.** The paper describes DtR as decompose → canonicalize → retrieve, but the released code has no canonicalization step and the paper publishes no prompts, so its 0.28 / 0.40 cannot be reproduced.
- **Reported per method**: identifier P / R / F1 and exact match per task and overall, candidate recall@k, IDs per answer, triple P / R / F1, RAGAS means on the 20 eval queries and the human ratings below.
- **Significance**: paired bootstrap 95% confidence intervals (10,000 resamples) on F1 and McNemar's exact test on exact match, for SGS vs VR and for each graph-embedding row vs the method it extends.

### RAGAS configuration

Setup: `scripts/ragas_eval.py`, RAGAS 0.4.3 collections metrics, judge gpt-5.4-mini-2026-03-17, embeddings text-embedding-3-large. Contexts are exactly what each method showed the model (SGS: the 5 evidence cards; VR: the 5 KB records at 600 characters). References are CTIConnect's reference answers, which also cover the questions' extra asks (likelihood of exploit, affected platforms, consequences).

### Human evaluation protocol

The author scores each system's answer on the 20 eval queries on a 1–5 scale, with answers shuffled and system labels hidden:

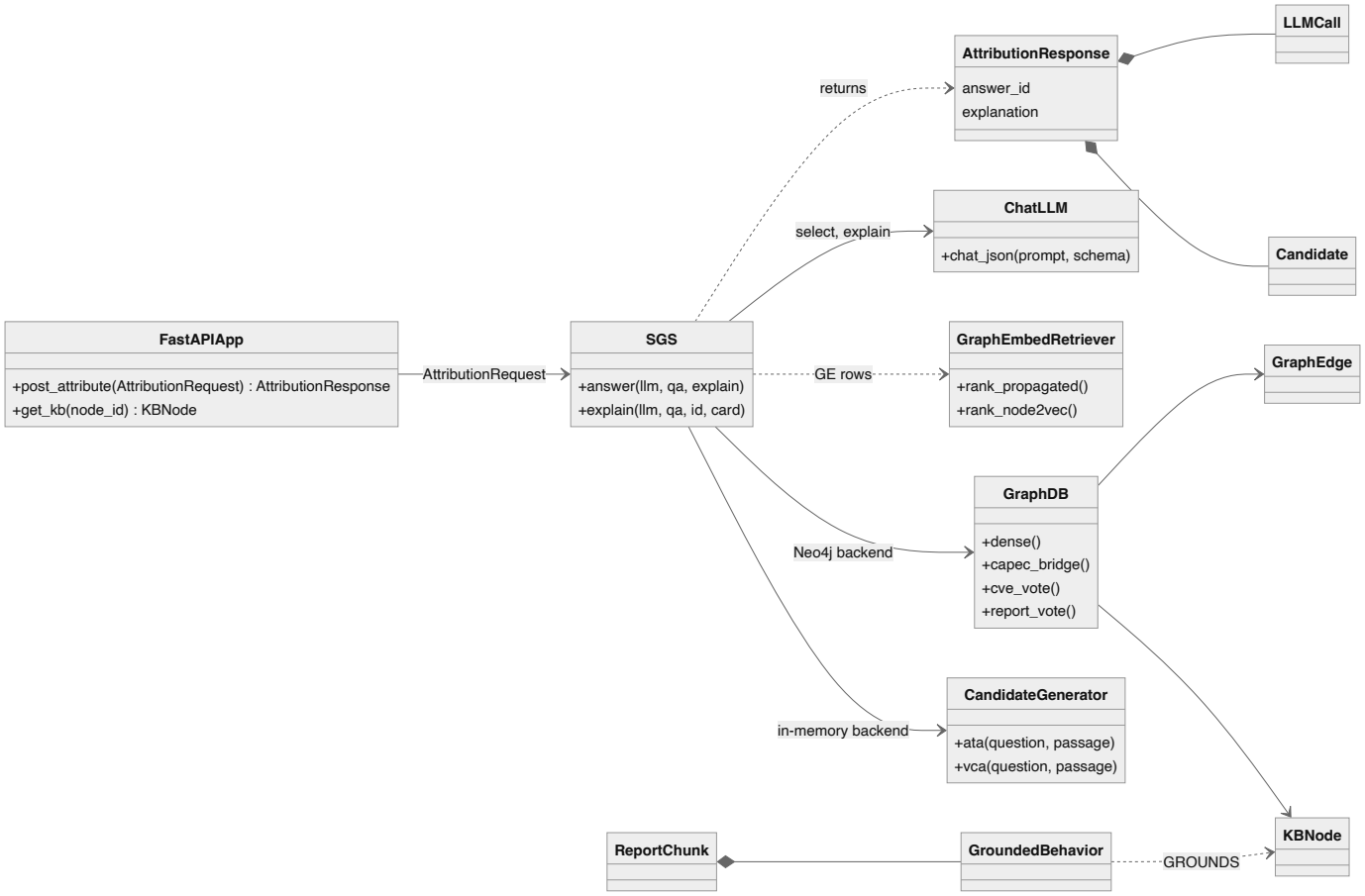
| Score | Factuality | Completeness | Coherence |
|---|---|---|---|
| 5 | ID and every claim match the KB entry and the passage | names the ID, explains the mapping, answers every sub-question | reasoning follows from passage to entry with no gaps |
| 4 | correct ID, one minor inaccuracy | one sub-question thin | one small gap |
| 3 | related ID (parent, child or sibling) or one wrong claim | one sub-question missing | understandable but loosely argued |
| 2 | wrong ID with a plausible argument | several parts missing | hard to follow |
| 1 | wrong ID or invented content | does not answer | incoherent |

As an individual project there is one rater; to check consistency, the author re-scores a random 10 answers a week later and reports quadratic-weighted Cohen's κ between the two passes.

## 8. Software Interfaces & Data Models

The engine is a Python package (`ctiattr`) behind a FastAPI service; a Streamlit chatbot calls it over HTTP, and the evaluation runner calls the same engine in-process so benchmark and chatbot answers come from one code path.

### Class diagram




The API calls the same `SGS` engine the evaluation runner uses; candidates come from the in-memory generators or from Cypher on Neo4j, and graph-embedding rankings feed the GE evaluation rows.

### Data models (`schemas.py`, Pydantic v2)

```
# src/ctiattr/schemas.py (excerpt; also the enums, Candidate, GroundedBehavior, GoldTriple, LLMCall)
ID_PATTERN = r"^(T\d{4}(\.\d{3})?|CWE-\d+|CAPEC-\d+|CVE-\d{4}-\d{4,})$"

class KBNode(BaseModel):                      # entity
    id: str = Field(..., pattern=ID_PATTERN)
    label: NodeLabel                          # AttackTechnique | CWE | CAPEC | CVE
    title: str
    text: str = ""
    tactics: list[str] = Field(default_factory=list)

class GraphEdge(BaseModel):                   # relation
    source_id: str
    target_id: str
    predicate: Predicate                      # the predicates of Section 3
    origin: str                               # "corpus_kb", "report_layer" or "rule:R<n>"

class ReportChunk(BaseModel):                 # graph chunk
    chunk_id: str = Field(..., pattern=r"^BLOG-\d+#\d+$")
```

```
    report_id: str = Field(..., pattern=r"^BLOG-\d+$")
    start: int = Field(..., ge=0)
    end: int = Field(..., gt=0)
    text: str

class AttributionRequest(BaseModel):          # query payload
    passage: str = Field(..., min_length=10)
    taxonomy: Taxonomy                        # attack (ATA) | cwe (VCA)
    question: str | None = None
    report_id: str | None = None              # excluded from report-layer examples
    method: str = "sgs"                       # "cb" | "vr" | "sgs"

class AttributionResponse(BaseModel):
    answer_id: str = Field(..., pattern=ID_PATTERN)   # exactly one identifier
    answer_title: str
    explanation: str = ""                     # no identifier other than answer_id
    candidates: list[Candidate] = Field(default_factory=list)
    calls: list[LLMCall] = Field(default_factory=list)
    latency_ms: float
```

### API endpoints (FastAPI)

| Method | Route | Request → response | Used by |
|---|---|---|---|
| POST | `/attribute` | `AttributionRequest` → `AttributionResponse` | chatbot, evaluation runner |
| GET | `/kb/{node_id}` | → `KBNode` with parent and children | chatbot "explain this entry" panel |
| GET | `/kb/search?q=&taxonomy=&k=` | → `list[Candidate]` | chatbot search box, retrieval diagnostics |
| GET | `/health` | → model name, Ollama digest, graph counts | deployment checks |

**Frontend.** The Streamlit page takes a pasted passage, shows the chosen ID with its KB definition, the graph path drawn as a small tree (tactic → technique → sub-technique, or passage → CAPEC/CVE → CWE), and an expandable list of the candidates and LLM calls behind the answer.

## 9. Risks & References

### Risks and mitigations

| Risk | Mitigation |
|---|---|
| Retrieval caps every method: the gold ID must reach the 5 candidates the model sees | snapshot bridges and graph embeddings fused by RRF (Section 5); recall@k reported beside every F1 |
| Small samples: on 20 questions one answer moves F1 by 0.05–0.10 | decisions made on the 73-question dev split; final claims only from the 306-question test split, with bootstrap CIs |
| Settings and prompts overfit the dev questions | small search grids reported in full; dev numbers labelled as such; the test split is run once at the end |
| Baseline drift from the paper (quantization, decoding defaults) | fp16 model, CTIConnect prompts verbatim, differences from the paper reported |

**Repository checkpoint.** github.com/shinPark43/CTI-GraphRAG: `schema.cypher` and the Neo4j loader, the extraction prompt and parser tests, `data/gold_triples.json` (69 triples) and `data/eval_queries.json` (20 queries); its README lists every component and how to run it.

### References


- Boyer, A., Dogdu, E., Choupani, R., Watson, J. S., Sanchez, D., & Ametu, A. (2024). Toward a unified cybersecurity knowledge graph: Leveraging ontologies and open data sources. *Recent Advances in Next-Generation Data Science*, Springer.
- Cheng, Y., Liu, Y., Li, C., Song, D., & Gao, P. (2026). CTIConnect: A benchmark for retrieval-augmented LLMs over heterogeneous cyber threat intelligence. *KDD '26*. arXiv:2510.11974; code and data

- Cheng, Y., Li, C., Cui, Q., Wang, L., Chen, Y., Ding, W., & Gao, P. (2026). CTIFoundry: An agent-native corpus scaffold for cyber threat intelligence. arXiv:2608.18613
- Cormack, G. V., Clarke, C. L. A., & Büttcher, S. (2009). Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. *SIGIR '09*.
- Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2024). RAGAS: Automated evaluation of retrieval augmented generation. *EACL 2024 (demos)*.
- Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*.